Day 2 Documentation - GPS to Clean Location Data

Intern: Likhitha N | **Role:** IoT

> **Recorded/Simulated GPS data - NOT LIVE GPS.**

Objective

Understand the data flow from a GPS/GNSS source to the ESP32 firmware, parse recorded NMEA sentences, validate the parsed position, and print clean location fields.

Data flow: GPS -> ESP32 -> Location Data -> Backend/Database

Development approach

The Wokwi ESP32-S3 simulator was used because physical hardware and a suitable GPS simulator component were unavailable (Wokwi has no GPS module and Proteus needs a licence). Recorded NMEA sentences were used as input. This is a parsing exercise, not a live GPS test.

NMEA sentence types

- **GGA** provides UTC time, latitude, longitude, fix quality, satellite count, HDOP and altitude.
- **RMC** provides UTC time, validity status (A = valid, V = void), latitude, longitude, speed (knots), course and date.

Example recorded inputs:

- GGA: `$GPGGA,123519,4807.038,N,01131.000,E,1,08,0.9,545.4,M,46.9,M,,*47`
- RMC: `$GPRMC,123519,A,4807.038,N,01131.000,E,022.4,084.4,230394,003.1,W*6A`

What happens between the GPS sensor, the ESP32 and the location data

1. The GPS module sends NMEA text sentences over UART, about once per second.
2. The ESP32 reads the characters and the TinyGPS++ library assembles complete sentences.
3. The checksum (XOR of the characters between `$` and `*`) is verified; a sentence that fails is discarded.
4. GGA and RMC fields are decoded. Latitude/longitude arrive as `ddmm.mmmm` and are converted to decimal degrees: `degrees + minutes / 60` (negative for S/W).
5. The fix status is checked (GGA fix quality above 0 and a valid location).
6. The firmware builds the clean location structure and prints it. With a valid fix it also prepares the JSON payload for the backend; with no fix, nothing is prepared.

Location data structure

Field	Example
Device ID	CRASH_ESP32_01
Latitude	48.117300
Longitude	11.516667
Timestamp (UTC)	1994-03-23T12:35:19Z
GPS status	VALID (fix quality 1)

Coordinates come from parsing the NMEA input; they are not hard-coded.

Parsing result (recorded sample)

- Latitude: 48.117300 deg
- Longitude: 11.516667 deg
- UTC time: 12:35:19
- Recorded date: 23 March 1994
- Fix quality in GGA: 1 (GPS fix)
- Satellites: 8
- HDOP: 0.90

These values belong to the recorded sample. They do not represent the current location or a live satellite fix.

What worked

The parser extracted the coordinates, time/date and fix information, and the Serial Monitor displayed a clean location summary. The final sketch also handles a no-fix record and rejects a sentence with a wrong checksum.

Problems and solutions

- **No GPS component in the simulator / Proteus needs a licence:** recorded NMEA sentences were used to test the parsing logic.
- **Two-digit year:** the sample date `230394` has the year `94`. TinyGPS++ reports it as 2094. The firmware now maps two-digit years 80-99 to 1980-1999 (GPS started in 1980) and 00-79 to 2000-2079, so it prints 1994. This is a general rule, not a fix for one sample. The earlier Day 2 screenshot still shows 2094 because it was taken before this change.

Limitations

No live GPS reception, physical UART wiring, antenna performance or real-world accuracy was tested. Recorded NMEA data only verifies the software parsing path.

Next steps

Continue with simulated IMU data processing, keep the recorded-data limitation explicit in every output, and prepare the location structure for sending to the backend.